

# Documentacao branch testeL - arquivos e codigo

## Documentacao da branch testeL - arquivos e codigo

### Contexto da anotacao

- Projeto local analisado: `/home/ludraxter/Projetos/Projeto_toytrace_grupo1`
- Branch analisada: `testeL`
- Estado Git observado: `testeL...origin/testeL`
- Commit observado: `edd2360` - criação da branch, execução do make e analise dos código em progresso
- Arquivos versionados na branch: 35
- Arquivos nao versionados ignorados nesta anotacao: `tests/__pycache__` e caches locais de teste.
- Estado funcional observado:
  - `./toytrace --help` funciona e mostra a sintaxe de uso.
  - `./toytrace trace -- ./tests/targets/hello_write` ainda falha com erro: `TODO`  
Semana 2: implementar `launch_tracee()`.
  - `make test` falha no teste unitario porque `student_pair_syscall()` ainda retorna `0` e nao parecia entrada/saida.

### Objetivo do projeto

`toytrace` e um projeto didatico de Sistemas Operacionais. A ideia e construir uma versao pequena de um monitor de chamadas de sistema, parecido com `strace`, usando `ptrace` no Linux x86\_64.

O programa final deve executar outro programa como processo filho, parar esse filho em cada entrada e saida de `syscall`, ler registradores, montar eventos, parear a entrada com a saida e imprimir uma linha legivel para cada `syscall`.

Fluxo conceitual esperado:

```
usuario
-> ./toytrace trace [opcoes] -- programa [args]
-> main()
-> parse_args()
-> trace_program()
-> fork/ptrace/waitpid/PTRACE_SYSCALL
-> trace_observer()
-> student_pair_syscall()
```

```
-> student_format_event()  
-> saida no terminal
```

## Como o programa funciona hoje

O binario ja compila, mas a parte principal do tracing ainda esta em TODO.

1. `main()` chama `parse_args()` para interpretar a CLI.
2. Se o comando for valido, `main()` monta `struct trace_state`.
3. `trace_program()` deveria iniciar o processo monitorado.
4. A primeira funcao chamada dentro de `trace_program()` e `launch_tracee()`.
5. Como `launch_tracee()` ainda so imprime um TODO e retorna `-1`, o tracing real para ali.
6. Por isso os testes de integracao falham antes de qualquer syscall ser observada.

## Conceitos de sintaxe C usados no projeto

- `#ifndef`, `#define`, `#endif`: guardas de header. Evitam que o mesmo `.h` seja incluído duas vezes no mesmo arquivo compilado.
- `#include`: importa declaracoes de headers do projeto ou da biblioteca C/Linux.
- `struct`: agrupa campos relacionados, como `struct syscall_event`.
- `typedef`: cria apelido para tipo. O projeto usa `typedef void (*trace_observer_fn) (...)` para definir um tipo de ponteiro de funcao.
- Ponteiro de funcao: permite passar uma callback para o runtime chamar quando observar uma syscall.
- `char **argv` e `char *const argv[]`: vetor de strings usado para argumentos de linha de comando e para `execvp`.
- `static` em funcoes `.c`: limita a visibilidade da funcao ao arquivo atual.
- `const`: informa que a funcao nao deve alterar aquele dado.
- `size_t`: tipo sem sinal usado para tamanhos de buffers.
- `pid_t`: tipo usado pelo Unix/Linux para identificadores de processo.
- `snprintf(buf, bufsz, ...)`: escreve texto formatado em um buffer respeitando o tamanho maximo.
- `memset(&obj, 0, sizeof(obj))`: zera memoria de uma struct.
- `return -1, 0, 1`: padrao usado para indicar erro, sucesso/parcial ou evento pronto, dependendo da funcao.

## Conceitos de Linux usados no projeto

- `fork()`: cria um processo filho copiando o processo atual.
- `PTTRACE_TRACEME`: chamado pelo filho para permitir que o pai o monitore.

- `raise(SIGSTOP)` : faz o filho parar voluntariamente para o pai configurar o tracing.
- `execvp()` / `execve()` : substitui o processo atual pelo programa alvo.
- `waitpid()` : pai espera mudancas de estado do filho, como parada, saida ou sinal.
- `PTRACE_SETOPTIONS` : configura opcoes do tracing.
- `PTRACE_O_TRACESYSGOOD` : marca paradas de syscall com bit `0x80` , facilitando diferenciar de outros sinais.
- `PTRACE_SYSCALL` : deixa o filho rodar ate a proxima entrada ou saida de syscall.
- `PTRACE_GETREGS` : le registradores do processo filho parado.
- `PTRACE_PEEKDATA` : le memoria do processo filho.
- Registradores x86\_64 para syscall:
  - `orig_rax` : numero da syscall.
  - `rax` : retorno da syscall, valido na saida.
  - `rdi` , `rsi` , `rdx` , `r10` , `r8` , `r9` : seis argumentos da syscall.

## Arquivos de configuracao, documentacao e exemplo

### **LICENSE.md**

Licenca MIT do projeto.

Funcao:

- Define permissoes legais para usar, copiar, modificar, publicar e distribuir o software.
- Informa que o software e fornecido sem garantia.

Como funciona:

- E um arquivo textual, nao participa da compilacao.
- Deve acompanhar copias substanciais do projeto.

Sintaxe:

- Texto Markdown simples.
- Nao ha sintaxe de codigo executavel.

### **README.md**

Documento principal do projeto.

Funcao:

- Explica o objetivo do `toytrace` .
- Mostra como compilar com `make` .

- Mostra como usar `./toytrace trace -- programa`.
- Explica o que o professor fornece e o que os estudantes devem implementar.
- Lista syscalls obrigatorias para formatacao especial.

Como funciona:

- Serve como guia de entrada para o grupo.
- Aponta que a primeira semana e de exploracao.
- Mostra que o erro esperado no inicio e `TODO` Semana 2: implementar `launch_tracee()`.

Sintaxe:

- Markdown.
- Usa blocos de codigo com comandos shell e exemplos de saida.
- Usa listas para separar responsabilidades.

## Makefile

Arquivo de automacao de build e testes.

Funcao:

- Compila o binario principal `toytrace`.
- Compila os programas alvo em `tests/targets/`.
- Compila e roda testes unitarios e de integracao.
- Remove artefatos com `make clean`.

Como funciona:

- `CC := gcc` : define o compilador.
- `CFLAGS := -Wall -Wextra -Wpedantic -std=c11 -g -O0 -Iinclude` : ativa warnings, C11, debug, sem otimizacao e inclui headers.
- `BIN := toytrace` : nome do executavel principal.
- `COMMON_SRCS` : fontes comuns sem `main.c`.
- `MAIN_SRCS` : `src/main.c` mais fontes comuns.
- `MAIN_OBJS := $(MAIN_SRCS:.c=.o)` : transforma nomes `.c` em `.o`.
- `TARGET_SRCS := $(wildcard tests/targets/*.c)` : encontra fontes de alvos de teste.
- `TARGET_BINS := $(TARGET_SRCS:.c=)` : remove extensao `.c` para gerar executaveis.

Alvos importantes:

- `all` : depende de `toytrace` e `targets`.
- `$(BIN)` : linka objetos em `toytrace`.

- `targets` : compila todos os programas de teste.
- `test-unit` : compila e roda `tests/unit/test_student` .
- `test-integration` : roda `pytest` .
- `test` : roda unidade e integracao.
- `clean` : remove binarios e objetos.

Sintaxe:

```
alvo: dependencias
      comando
```

Linhas de comando do Makefile precisam começar com tab. Variaveis sao acessadas com `$(NOME)` .

**examples/sample\_outputs/trace\_hello\_write.txt**

Exemplo de saida esperada aproximada depois da implementacao.

Funcao:

- Mostra o formato desejado de trace legivel.
- Ajuda a entender o produto final da Semana 5.

Como funciona:

- Nao e usado automaticamente pelo build.
- Serve como referencia manual.

Conteudo esperado:

```
execve("./tests/targets/hello_write", ...) = 0
write(1, 0x7ffd0000, 13) = 13
exit_group(0) = 0
```

Sintaxe:

- Texto puro.
- Cada linha representa uma syscall ja pareada e formatada.

**Headers em `include/`**

**`include/cli.h`**

Header da camada de linha de comando.

Funcao:

- Declara `struct trace_options`.
- Declara `parse_args()`.
- Declara `print_usage()`.

Como funciona:

- `struct trace_options` carrega o resultado do parser:
  - `char **target_argv` : ponteiro para o trecho de `argv` que representa o programa alvo.
  - `int raw_events` : flag booleana para ativar o modo cru.
- `parse_args()` preenche essa struct.
- `print_usage()` imprime ajuda.

Sintaxe:

```
struct trace_options {  
    char **target_argv;  
    int raw_events;  
};
```

`char **target_argv` significa ponteiro para vetor de strings. Isso combina com a assinatura de `execvp`.

## **`include/student_api.h`**

Header da API que os estudantes devem implementar.

Funcao:

- Define estruturas e prototipos da parte estudantil.
- E o contrato entre `main.c` e os arquivos em `src/student/`.

Como funciona:

- Inclui `syscall_event.h` e `trace_runtime.h`.
- Define constantes:
  - `MAX_FILTER_NAMES 32`
  - `MAX_SYSCALL_TRACKED 512`
- Define `struct syscall_pairer`, que guarda estado entre entrada e saida de syscall.
- Declara:
  - `student_pair_syscall()`

- `student_format_event()`
- `student_debug_raw_event()`

Sintaxe:

```
struct syscall_pairer {
    int has_entry;
    struct syscall_event entry;
};
```

Essa struct e um pequeno estado persistente. `has_entry` indica se ja existe uma entrada guardada. `entry` guarda a syscall de entrada ate chegar a saida.

## `include/syscall_event.h`

Define a estrutura central do projeto.

Funcao:

- Representar um evento de syscall observado pelo runtime.

Como funciona:

- Cada parada em syscall vira um `struct syscall_event`.
- Na entrada, os argumentos sao validos.
- Na saida, o retorno e valido.
- Depois do pareamento, a struct completa deve ter argumentos da entrada e retorno da saida.

Campos:

- `pid_t pid`: processo observado.
- `int entering`: 1 para entrada, 0 para saida.
- `long syscall_no`: numero da syscall.
- `long ret`: retorno, valido na saida.
- `unsigned long args[6]`: ate seis argumentos da syscall.

Sintaxe:

```
struct syscall_event {
    pid_t pid;
    int entering;
    long syscall_no;
    long ret;
```

```
    unsigned long args[6];  
};
```

`args[6]` é um vetor fixo. O acesso usa `ev->args[0]`, `ev->args[1]` etc.

## **`include/syscall_names.h`**

Header de conversao entre numero e nome de syscall.

Funcao:

- Declarar funcoes que traduzem syscall.

Como funciona:

- `syscall_name(long syscall_no)` recebe um numero e retorna string como `"write"`.
- `syscall_number_by_name(const char *name)` recebe nome e retorna numero, ou `-1`.

Sintaxe:

```
const char *syscall_name(long syscall_no);  
long syscall_number_by_name(const char *name);
```

`const char *` é ponteiro para string que a funcao nao deve modificar.

## **`include/trace_helpers.h`**

Header de helpers do processo monitorado.

Funcao:

- Declarar `read_child_string()`.

Como funciona:

- A funcao le uma string da memoria do processo filho usando `ptrace`.
- Sera importante para mostrar caminhos de `openat` e `execve`.

Sintaxe:

```
int read_child_string(pid_t pid, unsigned long addr, char *buf, size_t  
bufsz);
```

Parametros:



- `pid` : processo filho.
- `addr` : endereco no espaco de memoria do filho.
- `buf` : buffer local onde a string sera copiada.
- `bufsz` : tamanho maximo do buffer.

## `include/trace_runtime.h`

Header do runtime de tracing.

Funcao:

- Declarar a callback de observacao e a funcao principal do runtime.

Como funciona:

- `trace_program()` recebe o programa alvo e uma callback.
- Sempre que o runtime observar uma syscall, deve chamar `observer(ev, userdata)`.

Sintaxe:

```
typedef void (*trace_observer_fn)(const struct syscall_event *ev, void
*userdata);
```

Isso cria um tipo chamado `trace_observer_fn` para ponteiro de funcao. A funcao apontada recebe um evento e um ponteiro generico de contexto.

```
int trace_program(char *const argv[],
                  trace_observer_fn observer,
                  void *userdata);
```

`void *userdata` permite passar qualquer struct de estado para a callback.

## Codigo-fonte em `src/`

### `src/main.c`

Entrada principal do programa.

Funcao:

- Conecta CLI, runtime e funcoes dos estudantes.
- Define a callback que recebe eventos do runtime.

Como funciona:

1. Define `struct trace_state`, com:
  - `raw_events`: se o usuario quer eventos crus.
  - `pairer`: estado de pareamento.
2. `main()` chama `parse_args()`.
3. Se o parser pedir ajuda, retorna `0`.
4. Se o parser falhar, retorna `2`.
5. Chama `trace_program(opts.target_argv, trace_observer, &state)`.
6. `trace_program()` chama `trace_observer()` para cada evento.

Comportamento de `trace_observer()`:

- Se `raw_events` estiver ativo:
  - chama `student_debug_raw_event()`;
  - imprime a linha com `puts()`;
  - retorna.
- Se `raw_events` estiver desativado:
  - chama `student_pair_syscall()`;
  - se ainda nao tiver syscall completa, nao imprime nada;
  - quando tiver syscall completa, chama `student_format_event()`;
  - imprime a linha final.

Sintaxe importante:

```
static void trace_observer(const struct syscall_event *ev, void *userdata)
```

`static` deixa a funcao privada neste arquivo. `userdata` chega como `void *`, mas e convertido implicitamente para `struct trace_state *`.

```
struct trace_state state = {0};
```

Inicializa todos os campos com zero.

## **src/cli.c**

Parser da linha de comando.

Funcao:

- Validar argumentos do usuario.
- Separar opcoes do `toytrace` do programa alvo.
- Preencher `struct trace_options`.

Como funciona:

- `find_separator()` procura `--`.
- `print_usage()` imprime a ajuda.
- `parse_args()` aplica as regras:
  - sem argumentos ou `--help / -h`: mostra ajuda e retorna `1`;
  - primeiro comando precisa ser `trace`;
  - precisa existir `--` e depois dele um programa alvo;
  - antes de `--`, so aceita `--raw-events`;
  - depois de `--`, tudo vira `target_argv`.

Sintaxe importante:

```
for (i = 1; i < argc; i++) {  
    if (strcmp(argv[i], "--") == 0) {  
        return i;  
    }  
}
```

Loop percorre argumentos. `strcmp()` retorna `0` quando as strings sao iguais.

```
opts->target_argv = &argv[sep + 1];
```

Guarda um ponteiro para dentro do proprio `argv`, começando no programa alvo.

## **src/syscall\_names.c**

Tabela de nomes de syscalls.

Funcao:

- Converter numero de syscall para nome.
- Converter nome para numero.

Como funciona:

- Define `struct syscall_name_entry` com `no` e `name`.
- Cria `static const syscall_table[]`.
- Usa `#ifdef SYS_read`, `#ifdef SYS_write` etc para so incluir syscalls existentes na plataforma.
- `syscall_name()` percorre a tabela e retorna o nome.
- `syscall_number_by_name()` percorre a tabela e compara strings.

Sintaxe importante:

```
static const struct syscall_name_entry syscall_table[] = {  
#ifdef SYS_read  
    {SYS_read, "read"},  
#endif  
};
```

`#ifdef` é avaliado pelo pre-processador antes da compilação. Isso deixa o código mais portável entre sistemas que podem ter nomes de syscall diferentes.

## src/trace\_helpers.c

Helper para ler memória do processo filho.

Função:

- Implementar `read_child_string()`.
- Permitir que o tracer leia strings apontadas por argumentos de syscall do processo monitorado.

Como funciona:

1. Verifica se `bufsz == 0`; se sim, retorna erro.
2. Enquanto houver espaço no buffer:
  - chama `ptrace(PTRACE_PEEKDATA, pid, addr + copied, NULL)`;
  - lê uma palavra de memória do filho;
  - copia byte a byte para `buf`;
  - para quando encontra `'\0'`.
3. Se o buffer acabar antes da string terminar, fecha com `'\0'` e retorna `1`.
4. Se `ptrace` falhar, limpa `buf[0]` e retorna `-1`.

Retornos:

- `0` : string lida inteira.
- `1` : string truncada, mas buffer contém uma string terminada.
- `-1` : erro.

Sintaxe importante:

```
errno = 0;  
word = ptrace(PTRACE_PEEKDATA, pid, (void *) (addr + copied), NULL);  
if (word == -1 && errno != 0) {
```

```
    buf[0] = '\\0';  
    return -1;  
}
```

Como `ptrace` pode retornar `-1` como dado valido, o codigo zera `errno` antes e so considera erro se `errno` mudou.

## **src/trace\_runtime.c**

Runtime principal de tracing.

Funcao:

- Criar e controlar o processo monitorado.
- Esperar paradas de `syscall`.
- Ler registradores.
- Gerar `struct syscall_event`.
- Chamar a callback do observador.

Estado atual:

- Este e o arquivo mais importante para as Semanas 2, 3 e 4.
- Varias funcoes ainda estao como `TODO` e retornam erro.
- Por isso o projeto ainda nao faz tracing real.

Funcoes internas:

- `fill_event_from_regs()` : deve preencher `struct syscall_event` com dados dos registradores.
- `launch_tracee()` : deve criar o processo filho com `fork`, `PTRACE_TRACEME`, `SIGSTOP` e `execvp`.
- `wait_for_initial_stop()` : deve esperar o filho parar no `SIGSTOP` inicial.
- `configure_trace_options()` : deve ativar `PTRACE_O_TRACESYSGOOD`.
- `resume_until_next_syscall()` : deve chamar `PTRACE_SYSCALL`.
- `wait_for_syscall_stop()` : deve esperar a proxima parada e classificar se e `syscall`, fim ou erro.
- `trace_program()` : orquestra tudo.

Fluxo de `trace_program()`:

```
validar argv  
launch_tracee()  
wait_for_initial_stop()
```

```

configure_trace_options()
resume_until_next_syscall()
loop:
    wait_for_syscall_stop()
    se terminou: retornar codigo de saida
    PTRACE_GETREGS deveria preencher regs
    fill_event_from_regs()
    observer()
    alternar entering
    resume_until_next_syscall()

```

Sintaxe importante:

```

#if !defined(__x86_64__)
#error "Este runtime didatico suporta apenas Linux x86_64."
#endif

```

Impede compilacao fora de Linux x86\_64, porque os registradores esperados sao especificos dessa arquitetura.

```

int entering = 1;
entering = !entering;

```

Alterna entre evento de entrada e saida. Em `ptrace`, cada syscall gera duas paradas: uma antes de executar e outra depois.

TODOs por semana:

- Semana 2:
  - `launch_tracee()`
  - `wait_for_initial_stop()`
- Semana 3:
  - `configure_trace_options()`
  - `resume_until_next_syscall()`
  - `wait_for_syscall_stop()`
- Semana 4:
  - `fill_event_from_regs()`
  - chamada real de `PTRACE_GETREGS` no loop.

**Codigo dos estudantes em `src/student/`**

**`src/student/pairer.c`**

Arquivo de pareamento de eventos.

Funcao:

- Juntar a metade de entrada e a metade de saida da mesma syscall.
- Produzir um evento completo apenas quando ja existirem argumentos e retorno.

Estado atual:

- Ainda nao implementado.
- `student_pair_syscall()` sempre retorna `0`.
- Isso faz o teste unitario falhar, porque o teste espera retorno `1` no evento de saida.

Como deve funcionar:

1. Se `ev->entering == 1`:
  - copiar `*ev` para `pairer->entry`;
  - marcar `pairer->has_entry = 1`;
  - retornar `0`.
2. Se `ev->entering == 0`:
  - verificar se existe entrada pendente;
  - verificar se `pid` e `syscall_no` batem;
  - copiar a entrada para `*out`;
  - preencher `out->entering = 0`;
  - preencher `out->ret = ev->ret`;
  - limpar `has_entry`;
  - retornar `1`.
3. Se a sequencia for invalida:
  - retornar `-1`.

Sintaxe importante:

```
int student_pair_syscall(struct syscall_pairer *pairer,
                        const struct syscall_event *ev,
                        struct syscall_event *out)
```

`pairer` e ponteiro porque a funcao precisa modificar estado persistente. `ev` e `const` porque deve ser lido, nao alterado. `out` e ponteiro de saida.

**src/student/formatter.c**

Arquivo de formatacao dos eventos.

## Funcao:

- Converter `struct syscall_event` em texto para imprimir no terminal.
- Fornecer tambem uma formatacao crua para depuracao.

## Funcoes:

- `student_debug_raw_event()` : imprime formato simples para Semana 4.
- `student_format_event()` : imprime syscall completa em formato legivel.

## Estado atual:

- `student_debug_raw_event()` ja imprime `pid` , nome da syscall e se e entrada/saida.
- `student_format_event()` imprime formato generico:

```
nome(arg0, arg1, arg2, arg3, arg4, arg5) = retorno
```

## Ainda falta tratamento especial para:

- `read(fd, buf, count)`
- `write(fd, buf, count)`
- `openat(dirfd, "path", flags, mode)`
- `execve("path", ...)`
- `exit_group(status)`

## Sintaxe importante:

```
snprintf(buf, bufsz, "%s(%#lx, %#lx, %#lx, %#lx, %#lx, %#lx) = %ld",
        syscall_name(ev->syscall_no),
        ev->args[0],
        ev->args[1],
        ev->args[2],
        ev->args[3],
        ev->args[4],
        ev->args[5],
        ev->ret);
```

- `%s` : string.
- `%#lx` : hexadecimal longo com prefixo `0x` .
- `%ld` : inteiro longo decimal.
- `snprintf` evita estouro de buffer se usado corretamente.



# Testes e programas alvo

## tests/test\_integration.py

Testes de integracao em Python com `pytest`.

Funcao:

- Executar o binario real `toytrace`.
- Validar se a saida contem trechos esperados.

Como funciona:

- `ROOT = Path(__file__).resolve().parents[1]` encontra a raiz do projeto.
- `run_toytrace(args)` chama `subprocess.run()`.
- O teste falha se `proc.returncode != 0`.

Testes:

- `test_trace_hello_write_contains_write()`: espera que o trace de `hello_write` contenha `write( e = .`
- `test_raw_events_show_entry_and_exit()`: espera que `--raw-events` mostre entrada e saida.
- `test_trace_exec_echo_mentions_execve()`: espera que o trace de `exec_echo` mencione `execve(`.

Sintaxe importante:

```
proc = subprocess.run(..., cwd=ROOT, text=True,
                        stdout=subprocess.PIPE,
                        stderr=subprocess.PIPE,
                        check=False)
```

`check=False` impede execucao automatica; o teste faz o assert manualmente.

## tests/unit/test\_student.c

Testes unitarios em C das funcoes dos estudantes.

Funcao:

- Testar `student_pair_syscall()`.
- Testar `student_format_event()`.

Como funciona:

- Cria structs artificiais de syscall.
- Usa `assert()` para validar comportamento.
- Roda como executavel C separado.

Testes:

- `test_pair_write_entry_exit()`:
  - cria uma entrada de `SYS_write`;
  - cria uma saida da mesma syscall;
  - espera que a segunda chamada retorne `1`;
  - espera que `out` tenha argumentos da entrada e retorno da saida.
- `test_formatter_generic_mentions_return()`:
  - cria evento `SYS_close`;
  - chama `student_format_event()`;
  - verifica se a linha contem `close` e `= 0`.

Sintaxe importante:

```
struct syscall_event entry = {
    .pid = 123,
    .entering = 1,
    .syscall_no = SYS_write,
    .args = {1, 0xabc, 6, 0, 0, 0},
};
```

Isso e inicializacao designada de struct em C. Cada campo e preenchido pelo nome.

## tests/targets/hello\_write.c

Programa alvo simples.

Funcao:

- Gerar uma syscall `write`.
- Servir como alvo basico para o tracer.

Como funciona:

- Define a string `"ola toytrace\n"`.
- Chama `write(STDOUT_FILENO, msg, strlen(msg))`.
- Retorna `0`.

Sintaxe importante:

```
const char msg[] = "ola toytrace\n";
write(STDOUT_FILENO, msg, strlen(msg));
```

`STDOUT_FILENO` é o descritor de arquivo `1`. `strlen(msg)` calcula o tamanho da string sem contar `'\0'`.

## tests/targets/exec\_echo.c

Programa alvo que executa outro programa.

Funcao:

- Gerar uma syscall `execve`.
- Testar se o tracer consegue observar execucao de `/bin/echo`.

Como funciona:

- Monta `argv` com `"/bin/echo"` e `"eco"`.
- Monta `envp` vazio.
- Chama `execve("/bin/echo", argv, envp)`.
- Se `execve` der certo, o processo é substituído e o `return 1` não roda.
- Se `execve` falhar, retorna `1`.

Sintaxe importante:

```
char *const argv[] = {"/bin/echo", "eco", NULL};
char *const envp[] = {NULL};
```

Vetores passados para `execve` precisam terminar com `NULL`.

## tests/targets/failed\_open.c

Programa alvo que tenta abrir arquivo inexistente.

Funcao:

- Gerar uma syscall de abertura com erro.
- Testar como o tracer mostra retornos negativos.

Como funciona:

- Chama `open("/arquivo/que/nao/deve/existir/toytrace", O_RDONLY)`.
- Se `fd < 0`, retorna `0`, porque o erro era esperado.
- Se por algum motivo abriu, retorna `1`.

Sintaxe importante:

```
int fd = open("caminho", O_RDONLY);
return fd < 0 ? 0 : 1;
```

O operador ternario `condicao ? valor_se_verdadeiro : valor_se_falso` escolhe o retorno.

## tests/targets/open\_hosts.c

Programa alvo que abre e le `/etc/hosts`.

Funcao:

- Gerar syscalls de `open`, `read` e `close`.
- Servir para testar tracing de leitura de arquivo real.

Como funciona:

- Cria `char buf[32]`.
- Abre `/etc/hosts` somente leitura.
- Se falhar, retorna `1`.
- Le ate 32 bytes.
- Fecha o descritor.
- Retorna `0`.

Sintaxe importante:

```
char buf[32];
int fd = open("/etc/hosts", O_RDONLY);
read(fd, buf, sizeof(buf));
close(fd);
```

`sizeof(buf)` retorna o tamanho total do array local.

## Artefatos compilados versionados

Estes arquivos estao dentro da branch `testeL`, mas sao produtos de compilacao. Eles normalmente seriam gerados por `make` e removidos por `make clean`.

### toytrace

Executavel principal compilado.

Funcao:

- Binario ELF 64-bit que roda a CLI do projeto.
- Foi gerado a partir de `src/main.c` e dos fontes em `src/`.

Como funciona:

- Quando executado, entra em `main`.
- Aceita `--help`.
- Ao tentar trace real, chama codigo que ainda contem TODO.

Simbolos relevantes observados:

- `main`
- `parse_args`
- `trace_program`
- `read_child_string`
- `student_pair_syscall`
- `student_format_event`
- `student_debug_raw_event`

Sintaxe:

- Nao tem sintaxe de texto editavel; e um ELF compilado.
- Deve ser recriado com `make`.

## Objetos `.o` em `src/`

Arquivos:

- `src/cli.o`
- `src/main.o`
- `src/syscall_names.o`
- `src/trace_helpers.o`
- `src/trace_runtime.o`
- `src/student/formatter.o`
- `src/student/pairer.o`

Funcao:

- Cada `.o` e um objeto relocavel gerado pela compilacao de um `.c`.
- O linker junta esses objetos para formar `toytrace`.

Como funcionam:

- Contem codigo de maquina e simbolos.
- Ainda nao sao executaveis independentes.
- Guardam informacoes de debug porque o Makefile usa `-g`.

Sintaxe:

- Nao possuem sintaxe fonte editavel.
- A sintaxe correspondente esta nos arquivos `.c` de mesmo nome.

## Executaveis em `tests/targets/`

Arquivos:

- `tests/targets/hello_write`
- `tests/targets/exec_echo`
- `tests/targets/failed_open`
- `tests/targets/open_hosts`

Funcao:

- Binarios ELF usados como programas alvo do tracer.
- Cada um foi compilado a partir do `.c` correspondente.

Como funcionam:

- `hello_write`: chama `write`.
- `exec_echo`: chama `execve`.
- `failed_open`: chama `open` em caminho inexistente.
- `open_hosts`: chama `open`, `read` e `close`.

Sintaxe:

- Nao possuem sintaxe fonte editavel.
- A sintaxe correspondente esta nos arquivos `.c` com o mesmo nome.

## Mapa completo dos arquivos versionados

### Raiz

- `LICENSE.md`: licenca MIT.
- `Makefile`: build, testes, limpeza e alvos auxiliares.
- `README.md`: explicacao oficial do projeto.

- `toytrace` : executavel principal compilado.

## Exemplos

- `examples/sample_outputs/trace_hello_write.txt` : exemplo manual da saida final esperada.

## Headers

- `include/cli.h` : contrato do parser da CLI.
- `include/student_api.h` : contrato das funcoes implementadas pelos estudantes.
- `include/syscall_event.h` : struct central de evento de syscall.
- `include/syscall_names.h` : contrato da tabela numero/nome de syscall.
- `include/trace_helpers.h` : contrato de leitura de memoria do filho.
- `include/trace_runtime.h` : contrato do runtime e callback de observacao.

## Fontes principais

- `src/cli.c` : parse da linha de comando.
- `src/main.c` : entrada do programa e callback do trace.
- `src/syscall_names.c` : tabela de syscalls conhecidas.
- `src/trace_helpers.c` : leitura de string no processo filho.
- `src/trace_runtime.c` : runtime de `ptrace` , ainda com TODOs principais.

## Fontes dos estudantes

- `src/student/pairer.c` : deve parear entrada e saida de syscall; ainda nao implementado.
- `src/student/formatter.c` : formatacao final e raw; formato generico ja existe, especiais ainda faltam.

## Objetos compilados

- `src/cli.o`
- `src/main.o`
- `src/syscall_names.o`
- `src/trace_helpers.o`
- `src/trace_runtime.o`
- `src/student/formatter.o`
- `src/student/pairer.o`

## Testes e alvos

- `tests/test_integration.py` : testes pytest contra o binario real.
- `tests/unit/test_student.c` : testes unitarios em C.
- `tests/targets/hello_write.c` e `tests/targets/hello_write` : alvo de `write`.
- `tests/targets/exec_echo.c` e `tests/targets/exec_echo` : alvo de `execve`.
- `tests/targets/failed_open.c` e `tests/targets/failed_open` : alvo de abertura com erro.
- `tests/targets/open_hosts.c` e `tests/targets/open_hosts` : alvo de `open`, `read`, `close`.

## Onde começa o programa

O programa começa em:

```
src/main.c -> main()
```

`main()` é o ponto de entrada padrão de programas C. No Toytrace, ele não faz tracing diretamente; ele organiza o estado, chama o parser e delega para o runtime.

## Onde cria o processo

O processo deveria ser criado em:

```
src/trace_runtime.c -> launch_tracee()
```

Essa função ainda está em TODO. O fluxo esperado é:

```
fork()
  filho:
    ptrace(PTRACE_TRACEME)
    raise(SIGSTOP)
    execvp(argv[0], argv)
  pai:
    retorna pid do filho
```

## Onde chama o callback

A callback é chamada em:

```
src/trace_runtime.c -> trace_program()
```

Dentro do loop, depois de preencher `struct syscall_event`, o runtime faz:



```
if (observer != NULL) {  
    observer(&ev, userdata);  
}
```

No programa real, essa callback e:

```
src/main.c -> trace_observer()
```

## Onde devemos mexer

Principais arquivos de implementacao:

- `src/trace_runtime.c`: Semanas 2, 3 e 4.
- `src/student/pairer.c`: pareamento de eventos.
- `src/student/formatter.c`: formatacao final das syscalls.

Arquivos que devem ser lidos com cuidado, mas normalmente nao precisam mudar no inicio:

- `src/main.c`
- `src/cli.c`
- `include/*.h`
- `tests/unit/test_student.c`
- `tests/test_integration.py`

## Ordem recomendada de estudo e implementacao

1. Entender `README.md`.
2. Rodar `./toytrace --help`.
3. Ler `src/main.c`.
4. Ler `src/cli.c`.
5. Ler `include/syscall_event.h` e `include/trace_runtime.h`.
6. Ler `src/trace_runtime.c`.
7. Implementar Semana 2:
  - `launch_tracee()`
  - `wait_for_initial_stop()`
8. Implementar `student_pair_syscall()` para passar o teste unitario.
9. Implementar Semana 3:
  - `configure_trace_options()`
  - `resume_until_next_syscall()`
  - `wait_for_syscall_stop()`

10. Implementar Semana 4:

- `PTRACE_GETREGS`
- `fill_event_from_regs()`

11. Melhorar `student_debug_raw_event()` .

12. Implementar formatacao especial em `student_format_event()` .

13. Rodar `make test` .

## Diagnostico atual da branch

O projeto esta no estado esperado de inicio/analise:

- CLI ja funciona.
- Build base existe.
- Testes ja estao configurados.
- Programas alvo ja existem.
- Runtime ainda nao cria processo.
- Pairer ainda nao junta entrada/saida.
- Formatter ainda e generico.

Primeiro TODO que bloqueia execucao real:

```
src/trace_runtime.c -> launch_tracee()
```

Primeiro teste unitario que bloqueia:

```
tests/unit/test_student.c -> test_pair_write_entry_exit()
```

Esse teste deixa claro o comportamento minimo esperado de `student_pair_syscall()` .